

102B Final Project

Daren Sathasivam – 306229580

2024-06-14

Problem 1:

Consider a random variable X from a Weibull distribution. This distribution is extensively used in material science and industrial statistics, since it can model well the “aging” process of materials and components on physical systems.

Hint: To generate a random sample s of size n from a Weibull distribution with shape parameter k and scale parameter λ use the following command:

```
# s = rweibull(n, shape = k, scale = lambda)
```

Part (a):

Evaluate the **convergence probability** and **average length** of the following **95% confidence intervals** for the **population median** based on random samples of size $n = 100$ and 250 , drawn from a **Weibull distribution with shape parameter $k = 1.5$ and scale parameter $\lambda = 1$** .

Your results should be based on $M = 100, 500$ random samples and $B = 5000$ bootstrap replicates.

```
set.seed(100)
# Load helper functions
source("bootsample.R")
source("bootstats.R")
library(boot)

# Set parameters
n_sizes <- c(100, 250)
M_values <- c(100, 500)
k <- 1.5
lambda <- 1
B <- 5000
alpha <- 0.05
true_median <- lambda * (log(2))^(1/k)

# Function to generate Weibull samples
generate_sample <- function(n) {
  rweibull(n, shape = k, scale = lambda)
}

# Function to calculate medians for bootstrap samples
median_fun <- function(data) {
  median(data)
}

# Function to calculate confidence intervals
```

```

calculate_cis <- function(sample) {
  bootstrap_samples <- bootstrapping(sample, B)
  boot_results <- boot.stats(bootstrap_samples, median_fun)

  t0 <- median(sample)
  se <- boot_results$se
  z <- qnorm(1 - alpha/2)

  # Normal CI
  normal_ci <- c(t0 - z * se, t0 + z * se)
  # Basic CI
  basic_ci <- c(2 * t0 - quantile(boot_results$theta, 1 - alpha/2),
    2 * t0 - quantile(boot_results$theta, alpha/2))
  # Percentile CI
  percentile_ci <- quantile(boot_results$theta, c(alpha/2,
    1 - alpha/2))
  # Studentized CI
  studentized_ci <- c(t0 - z * se, t0 + z * se)

  return(list(normal = normal_ci, basic = basic_ci, percentile = percentile_ci,
    studentized = studentized_ci))
}

# Simulation function to evaluate CIs
evaluate_cis <- function(n, M) {
  coverage <- matrix(0, ncol = 4, nrow = M)
  lengths <- matrix(0, ncol = 4, nrow = M)
  last_cis <- NULL

  for (i in 1:M) {
    sample <- generate_sample(n)
    cis <- calculate_cis(sample)

    for (j in seq_along(cis)) {
      ci <- cis[[names(cis)[j]]]
      # Check if true median is within CI
      coverage[i, j] <- ci[1] <= true_median && ci[2] >=
        true_median
      # Get length of CI
      lengths[i, j] <- ci[2] - ci[1]
    }
    last_cis <- cis
  }

  # Print CIs
  cat("\nConfidence Intervals for n =", n, "and M =", M, ":\n")
  for (ci_name in names(last_cis)) {
    ci <- last_cis[[ci_name]]
    cat(ci_name, "CI: [", format(ci[1], digits = 4), ", ",
      format(ci[2], digits = 4), "]\n")
  }

  # Average out
  list(coverage = colMeans(coverage), length = colMeans(lengths))
}

```

```

}

# Place results into table
results <- matrix(nrow = 8, ncol = 4)
colnames(results) <- c("Normal", "Basic", "Percentile", "Studentized")
rownames(results) <- c("Coverage (n=100, M=100)", "Length (n=100, M=100)",
  "Coverage (n=100, M=500)", "Length (n=100, M=500)", "Coverage (n=250, M=100)",
  "Length (n=250, M=100)", "Coverage (n=250, M=500)", "Length (n=250, M=500)")

index <- 1
for (n in n_sizes) {
  for (M in M_values) {
    sim_results <- evaluate_cis(n, M)
    results[index, ] <- sim_results$coverage
    results[index + 1, ] <- sim_results$length
    index <- index + 2
  }
}

```

```

##
## Confidence Intervals for n = 100 and M = 100 :
## normal CI: [ 0.7295 , 0.9618 ]
## basic CI: [ 0.7694 , 1.014 ]
## percentile CI: [ 0.6778 , 0.9219 ]
## studentized CI: [ 0.7295 , 0.9618 ]
##
## Confidence Intervals for n = 100 and M = 500 :
## normal CI: [ 0.6794 , 0.9402 ]
## basic CI: [ 0.7016 , 0.9455 ]
## percentile CI: [ 0.6741 , 0.918 ]
## studentized CI: [ 0.6794 , 0.9402 ]
##
## Confidence Intervals for n = 250 and M = 100 :
## normal CI: [ 0.7399 , 1.009 ]
## basic CI: [ 0.7664 , 1.009 ]
## percentile CI: [ 0.7395 , 0.9821 ]
## studentized CI: [ 0.7399 , 1.009 ]
##
## Confidence Intervals for n = 250 and M = 500 :
## normal CI: [ 0.7355 , 0.8857 ]
## basic CI: [ 0.7166 , 0.8723 ]
## percentile CI: [ 0.7489 , 0.9046 ]
## studentized CI: [ 0.7355 , 0.8857 ]

```

```

# Output results
print(results)

```

```

##
## Coverage (n=100, M=100) 1.0000000 0.9500000 0.9600000 1.0000000
## Length (n=100, M=100) 0.2956493 0.2911314 0.2911314 0.2956493
## Coverage (n=100, M=500) 0.9340000 0.8520000 0.9300000 0.9340000
## Length (n=100, M=500) 0.2993562 0.2935129 0.2935129 0.2993562
## Coverage (n=250, M=100) 0.9300000 0.8700000 0.9400000 0.9300000
## Length (n=250, M=100) 0.1862694 0.1829907 0.1829907 0.1862694
## Coverage (n=250, M=500) 0.9420000 0.9100000 0.9420000 0.9420000

```

```
## Length (n=250, M=500)    0.1862819 0.1843376 0.1843376    0.1862819
```

Provide a description of your simulation focusing on how you calculated the coverage probability and average length of each type of confidence interval, a summary of your findings with comments, and your R code and output.

Your findings should be organized in a table, where the columns correspond to the different types of bootstrap confidence intervals and the rows to the coverage probability and the average length for each of the n, M combination. Hence, your table should have 8 rows and 4 columns with clear description of each one.

- It was given that the true median of the Weibull function is $median(X) = \lambda [\log(2)]^{\frac{1}{k}}$. Using this, the simulation was designed to test each of the four types of bootstrap confidence intervals with the Weibull distribution with the shape parameter $k = 1.5$ and scale parameter $\lambda = 1$. The Normal, Basic, Percentile, and Studentized bootstrap calculations were calculated using the examples from week 1 and week 2 lectures. The coverage probability was calculated by the proportion of confidence intervals that contained the true median across all simulations. The average length was calculated as the mean of the widths of each type of confidence interval across all simulations. The coverage probabilities for the sample sizes of 100 were clearly smaller than the sample sizes of 250, which indicates that the confidence intervals are more accurate with larger sample sizes. Additionally, it can be observed that the percentile bootstrap CIs had a relatively larger coverage probability, indicating that the percentile bootstrap CI may be the most efficient for this simulation. The average lengths among the four types are relatively similar. However, as the sample size increased, it can be observed that the average lengths decreased, indicating that the larger sample size resulted in more precise estimates.

Part(b):

Evaluate the **convergence probability** and **average length** of the following **95% confidence intervals** for the **population standard deviation** based on random samples of size $n = 100$ and 250 , drawn from a **Weibull distribution with shape parameter $k = 1.5$ and scale parameter $\lambda = 1$** .

Your results should be based on $M = 100, 500$ random samples and $B = 5000$ bootstrap replicates.

```
set.seed(100)
# Load helper functions
source("bootsample.R")
source("bootstats.R")
library(boot)

# Set parameters
n_sizes <- c(100, 250)
M_values <- c(100, 500)
k <- 1.5
lambda <- 1
B <- 5000
alpha <- 0.05
true_sd <- sqrt(lambda^2 * (gamma(1 + 2/k) - (gamma(1 + 1/k))^2))

# Function to generate Weibull samples
generate_sample <- function(n) {
  rweibull(n, shape = k, scale = lambda)
}

# Function to calculate standard deviation for bootstrap
# samples
```

```

sd_fun <- function(data) {
  sd(data)
}

# Function to calculate confidence intervals
calculate_cis <- function(sample) {
  bootstrap_samples <- bootstrapping(sample, B)
  boot_results <- boot.stats(bootstrap_samples, sd_fun)

  t0 <- sd(sample)
  se <- boot_results$se
  z <- qnorm(1 - alpha/2)

  # Normal CI
  normal_ci <- c(t0 - z * se, t0 + z * se)
  # Basic CI
  basic_ci <- c(2 * t0 - quantile(boot_results$theta, 1 - alpha/2),
    2 * t0 - quantile(boot_results$theta, alpha/2))
  # Percentile CI
  percentile_ci <- quantile(boot_results$theta, c(alpha/2,
    1 - alpha/2))
  # Studentized CI
  studentized_ci <- c(t0 - z * se, t0 + z * se)

  return(list(normal = normal_ci, basic = basic_ci, percentile = percentile_ci,
    studentized = studentized_ci))
}

# Simulation function to evaluate CIs
evaluate_cis <- function(n, M) {
  coverage <- matrix(0, ncol = 4, nrow = M)
  lengths <- matrix(0, ncol = 4, nrow = M)
  last_cis <- NULL

  for (i in 1:M) {
    sample <- generate_sample(n)
    cis <- calculate_cis(sample)

    for (j in seq_along(cis)) {
      ci <- cis[[names(cis)[j]]]
      # Check if true median is within CI
      coverage[i, j] <- ci[1] <= true_sd && ci[2] >= true_sd
      # Get length of CI
      lengths[i, j] <- ci[2] - ci[1]
    }
    last_cis <- cis
  }

  # Print CIs
  cat("\nConfidence Intervals for n =", n, "and M =", M, ":\n")
  for (ci_name in names(last_cis)) {
    ci <- last_cis[[ci_name]]
    cat(ci_name, "CI: [", format(ci[1], digits = 4), ", ",
      format(ci[2], digits = 4), "]\n")
  }
}

```

```

}

# Average out
list(coverage = colMeans(coverage), length = colMeans(lengths))
}

# Place results into table
results <- matrix(nrow = 8, ncol = 4)
colnames(results) <- c("Normal", "Basic", "Percentile", "Studentized")
rownames(results) <- c("Coverage (n=100, M=100)", "Length (n=100, M=100)",
  "Coverage (n=100, M=500)", "Length (n=100, M=500)", "Coverage (n=250, M=100)",
  "Length (n=250, M=100)", "Coverage (n=250, M=500)", "Length (n=250, M=500)")

index <- 1
for (n in n_sizes) {
  for (M in M_values) {
    sim_results <- evaluate_cis(n, M)
    results[index, ] <- sim_results$coverage
    results[index + 1, ] <- sim_results$length
    index <- index + 2
  }
}

```

```

##
## Confidence Intervals for n = 100 and M = 100 :
## normal CI: [ 0.4608 , 0.6007 ]
## basic CI: [ 0.465 , 0.6049 ]
## percentile CI: [ 0.4566 , 0.5965 ]
## studentized CI: [ 0.4608 , 0.6007 ]
##
## Confidence Intervals for n = 100 and M = 500 :
## normal CI: [ 0.578 , 0.7962 ]
## basic CI: [ 0.587 , 0.7993 ]
## percentile CI: [ 0.5749 , 0.7872 ]
## studentized CI: [ 0.578 , 0.7962 ]
##
## Confidence Intervals for n = 250 and M = 100 :
## normal CI: [ 0.5547 , 0.6877 ]
## basic CI: [ 0.5565 , 0.6888 ]
## percentile CI: [ 0.5537 , 0.6859 ]
## studentized CI: [ 0.5547 , 0.6877 ]
##
## Confidence Intervals for n = 250 and M = 500 :
## normal CI: [ 0.5404 , 0.655 ]
## basic CI: [ 0.542 , 0.6585 ]
## percentile CI: [ 0.537 , 0.6534 ]
## studentized CI: [ 0.5404 , 0.655 ]

```

```

# Output results
print(results)

```

```

##
##           Normal      Basic Percentile Studentized
## Coverage (n=100, M=100) 0.9000000 0.8800000 0.8900000 0.9000000
## Length (n=100, M=100)  0.2077656 0.2060717 0.2060717 0.2077656
## Coverage (n=100, M=500) 0.9040000 0.8980000 0.9020000 0.9040000

```

```
## Length (n=100, M=500) 0.2039737 0.2026905 0.2026905 0.2039737
## Coverage (n=250, M=100) 0.9500000 0.9600000 0.9500000 0.9500000
## Length (n=250, M=100) 0.1341566 0.1338959 0.1338959 0.1341566
## Coverage (n=250, M=500) 0.9360000 0.9440000 0.9320000 0.9360000
## Length (n=250, M=500) 0.1359330 0.1355600 0.1355600 0.1359330
```

Provide a description of your simulation focusing on how you calculated the coverage probability and average length of each type of confidence interval, a summary of your findings with comments, and your R code and output.

Your findings should be organized in a table, where the columns correspond to the different types of bootstrap confidence intervals and the rows to the coverage probability and the average length for each of the n, M combination. Hence, your table should have 8 rows and 4 columns with clear description of each one.

- Using the same parameters as part a, I have simulated the confidence intervals in regards to the population standard deviation rather than the true median of the Weibull distribution. I utilized the same approach as part a for the calculations of coverage probability and average length of the confidence intervals but have replaced the value of interest from the median to the standard deviation. It can be observed that the coverage probabilities were relatively high across all four types of bootstrap. However, as the sample size increased, the coverage probability also increased, indicating that the larger sample size provides more accurate estimates. The average length of the CIs were relatively similar across all four types, however the larger sample size resulted in shorter lengths. This indicates that the larger sample size was more effective in capturing the variability within the data. It also can be noted that the Normal and Studentized seem to result in better coverage probabilities at smaller sample sizes but the Basic bootstrap confidence interval seemed to perform better than the rest at larger sample sizes.

Problem 2:

Consider the following regression model with n observations and p predictor variables, given by: $y_i = x_i^\top \beta + \epsilon_i, i = 1, \dots, n$,

where $y_i \in \mathbb{R}$ contains the i -th observation of the response variable, $x_i \in \mathbb{R}^p$ is a p -dimensional vector containing the i -th observation for the predictors.

Note that this regression model **does not contain an intercept term**.

Part (a):

Generate a data set of size $n = 100000$ for $p = 20$ predictors, where: - 1. all entries of the true regression coefficient β_{true} are equal to 3 - 2. all p predictors have mean 0 and their correlation matrix is determined by the following R commands:

```
# R = matrix(c(rep(0.99, p/2), rep(0.9, p/2)), p, p)
# diag(R) = 1 R = (R + t(R)) / 2
```

- 3. the error term is normally distributed, with mean 0 and **standard deviation 1.25**.

```
library(MASS)
# Set parameters
n <- 1e+05
p <- 20
beta_true <- rep(3, p)
sigma <- 1.25
# Define the correlation matrix
R <- matrix(c(rep(0.99, p/2), rep(0.9, p/2)), p, p)
diag(R) <- 1
R <- (R + t(R))/2
```

```

# Generate the predictor variables X
set.seed(100)
design <- mvrnorm(n, mu = rep(0, p), Sigma = R)
# Generate the error terms
error.term <- rnorm(n, mean = 0, sd = sigma)
# Generate the response variable y
response <- design %*% beta_true + error.term
# Combining X and y into a data frame
data <- as.data.frame(cbind(response, design))

names(data) <- c("y", paste("X", 1:p, sep = ""))

```

Part (b):

Estimate the regression coefficient $\hat{\beta}$ using the gradient descent algorithm and compare the estimate to the one obtained by the least squares method.

```

# Gradient Descent: Define the gradient
gradient <- function(X, y, beta) {
  -2 * t(X) %*% (y - X %*% beta)/nrow(X)
}
# Gradient descent function for regression
gradient_descent <- function(X, y, beta_init, stepsize, max_iter,
  tolerance) {
  beta <- beta_init
  n <- nrow(X)

  for (i in 1:max_iter) {
    # Calculate the gradient
    grad <- gradient(X, y, beta)

    # Update the coefficients
    beta_new <- beta - stepsize * grad

    # Check for convergence
    if (sqrt(sum((beta_new - beta)^2)) < tolerance) {
      cat("Converged at iteration:", i, "\n")
      break
    }
    beta <- beta_new
  }

  return(beta)
}

beta_init <- rep(0, p)
my_stepsize <- 0.01
max_iter <- 16000
tolerance <- 1e-06

# Run gd
estimated_beta <- gradient_descent(design, response, beta_init,
  my_stepsize, max_iter, tolerance)

```

```
## Converged at iteration: 15514
```

```
print(estimated_beta)
```

```
##           [,1]
## [1,] 3.017811
## [2,] 3.049351
## [3,] 2.972613
## [4,] 3.009447
## [5,] 3.034480
## [6,] 2.952420
## [7,] 2.980556
## [8,] 3.057407
## [9,] 2.976757
## [10,] 2.988141
## [11,] 2.970279
## [12,] 2.985516
## [13,] 2.994108
## [14,] 3.013545
## [15,] 2.990069
## [16,] 2.993590
## [17,] 3.011927
## [18,] 2.994940
## [19,] 3.004692
## [20,] 3.002444
```

```
# OLS:
```

```
X <- design
y <- response
lm_model <- lm(y ~ X - 1)
summary(lm_model)
```

```
##
## Call:
## lm(formula = y ~ X - 1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -5.4051 -0.8580 -0.0081  0.8339  5.0705
##
## Coefficients:
##      Estimate Std. Error t value Pr(>|t|)
## X1   3.01845    0.03800   79.43  <2e-16 ***
## X2   3.05138    0.03777   80.79  <2e-16 ***
## X3   2.97118    0.03772   78.76  <2e-16 ***
## X4   3.00969    0.03793   79.35  <2e-16 ***
## X5   3.03589    0.03767   80.58  <2e-16 ***
## X6   2.94996    0.03777   78.11  <2e-16 ***
## X7   2.97949    0.03773   78.97  <2e-16 ***
## X8   3.06005    0.03793   80.68  <2e-16 ***
## X9   2.97541    0.03790   78.50  <2e-16 ***
## X10  2.98746    0.03771   79.23  <2e-16 ***
## X11  2.97028    0.01262  235.30  <2e-16 ***
## X12  2.98552    0.01264  236.12  <2e-16 ***
## X13  2.99411    0.01262  237.26  <2e-16 ***
```

```
## X14  3.01354      0.01257  239.68   <2e-16 ***
## X15  2.99007      0.01270  235.36   <2e-16 ***
## X16  2.99360      0.01260  237.60   <2e-16 ***
## X17  3.01194      0.01261  238.76   <2e-16 ***
## X18  2.99494      0.01260  237.68   <2e-16 ***
## X19  3.00469      0.01260  238.50   <2e-16 ***
## X20  3.00245      0.01256  238.98   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.252 on 99980 degrees of freedom
## Multiple R-squared:  0.9995, Adjusted R-squared:  0.9995
## F-statistic: 1.086e+07 on 20 and 99980 DF,  p-value: < 2.2e-16
```

Further, discuss how you selected the step size, and comment on the number of iterations required and the accuracy of the estimated regression coefficients compared to the least squares solution.

- I have selected a stepsize of 0.01 as it balances the speed and accuracy of the convergence. I also have set a maximum number of iterations as 30000 with a tolerance of $1e-6$, but the algorithm is observed to converge after roughly 15000 iterations. The maximum number of iterations allows the gradient descent to near convergence with the set tolerance. However, if we would like more accurate results at the cost of convergence speed, we can set the maximum number of iterations to a larger value and the tolerance parameter to a smaller value. In comparison to the least squares solution, it can be seen that gradient descent has slightly more variability but remains close to the true beta value of 3. In the OLS method, it can be observed that all predictors are close to the true values with great accuracy which is reflected by the low standard error and high t-values.

Part (c):

Note that by construction, the predictors are very highly correlated and hence the condition number of the $X^T X$ matrix is large.

In such settings, it is known that a momentum adjustment of the gradient descent algorithm is helpful. Use the gradient descent algorithm with Polyak's heavy ball method and Nesterov's momentum method to obtain an estimate of the regression coefficients and compare it to your answers in Part (b).

```
# Polyak GD
polyakGD <- function(X, y, beta_init, stepsize, gamma, max_iter,
  tolerance) {
  beta <- beta_init
  beta_prev <- beta_init # Previous beta for momentum term
  n <- nrow(X)

  for (i in 1:max_iter) {
    grad <- gradient(X, y, beta)
    # from hw5
    beta_new <- beta - stepsize * grad + gamma * (beta -
      beta_prev)

    # Convergence check
    if (sqrt(sum((beta_new - beta)^2)) < tolerance) {
      cat("Convergence achieved at iteration", i, "\n")
      break
    }

    beta_prev <- beta
  }
}
```

```

    beta <- beta_new
  }

  return(list(beta = beta, iterations = i))
}

# Nesterov GD
nesterovGD <- function(X, y, beta_init, stepsize, gamma, max_iter,
  tolerance) {
  beta <- beta_init
  v <- rep(0, length(beta_init))
  n <- nrow(X)

  for (i in 1:max_iter) {
    # from hw5
    beta_lookahead <- beta - gamma * v
    grad <- gradient(X, y, beta_lookahead)

    v_new <- gamma * v + stepsize * grad
    beta_new <- beta - v_new

    if (sqrt(sum((beta_new - beta)^2)) < tolerance) {
      cat("Convergence achieved at iteration", i, "\n")
      break
    }

    v <- v_new
    beta <- beta_new
  }

  return(list(beta = beta, iterations = i))
}

beta_init <- rep(0, p)
my_stepsize <- 0.01
max_iter <- 3000
tolerance <- 1e-06
gamma <- 0.9 # Using from hw5 - tau not needed if no bls

results_polyak <- polyakGD(design, response, beta_init, my_stepsize,
  gamma, max_iter, tolerance)

## Convergence achieved at iteration 2673
results_nesterov <- nesterovGD(design, response, beta_init, my_stepsize,
  gamma, max_iter, tolerance)

## Convergence achieved at iteration 2677
# names(results_polyak) <- paste('X', 1:20, sep='')
# names(results_nesterov) <- paste('X', 1:20, sep='')
cat("Polyak's Heavy Ball Results:\n")

## Polyak's Heavy Ball Results:
print(results_polyak)

## $beta

```

```
##          [,1]
## [1,] 3.018390
## [2,] 3.051183
## [3,] 2.971321
## [4,] 3.009673
## [5,] 3.035754
## [6,] 2.950204
## [7,] 2.979589
## [8,] 3.059791
## [9,] 2.975540
## [10,] 2.987518
## [11,] 2.970277
## [12,] 2.985524
## [13,] 2.994108
## [14,] 3.013538
## [15,] 2.990067
## [16,] 2.993601
## [17,] 3.011935
## [18,] 2.994938
## [19,] 3.004693
## [20,] 3.002449
##
## $iterations
## [1] 2673
```

```
cat("\nNesterov's Momentum Results:\n")
```

```
##
## Nesterov's Momentum Results:
```

```
print(results_nesterov)
```

```
## $beta
##          [,1]
## [1,] 3.018390
## [2,] 3.051182
## [3,] 2.971322
## [4,] 3.009672
## [5,] 3.035754
## [6,] 2.950205
## [7,] 2.979589
## [8,] 3.059790
## [9,] 2.975540
## [10,] 2.987519
## [11,] 2.970277
## [12,] 2.985524
## [13,] 2.994108
## [14,] 3.013538
## [15,] 2.990067
## [16,] 2.993601
## [17,] 3.011935
## [18,] 2.994938
## [19,] 3.004693
## [20,] 3.002449
##
```

```
## $iterations
## [1] 2677

beta_ols <- coef(lm_model)
names(beta_ols) <- paste("X", 1:20, sep = "")
cat("OLS Results:\n")
```

```
## OLS Results:
```

```
print(beta_ols)
```

```
##      X1      X2      X3      X4      X5      X6      X7      X8
## 3.018453 3.051377 2.971181 3.009692 3.035893 2.949962 2.979487 3.060055
##      X9      X10     X11     X12     X13     X14     X15     X16
## 2.975406 2.987456 2.970277 2.985524 2.994108 3.013538 2.990066 2.993602
##      X17     X18     X19     X20
## 3.011936 2.994937 3.004694 3.002450
```

Specifically, discuss how you selected the step size, and comment on the number of iterations required and the accuracy of the estimated regression coefficients compared to the least squares solution and those obtained by the standard gradient descent algorithm used in Part(b).

- For part (c), I decided to use the same step size from part (b) to maintain both computational efficiency and speed. Additionally, I have selected the maximum number of iterations as 3000 and a tolerance of $1e-6$ to also facilitate the efficiency and accuracy of the results as the algorithms both converge at roughly 2700 iterations. It can be observed that both the Polyak's Heavy Ball method and Nesterov's momentum method result in estimated regression coefficients that are very similar to the OLS method. This indicates that both methods are quicker and more accurate in convergence due to the momentum terms which accelerate convergence when compared to the standard gradient descent algorithm.

Part (d):

Given the large number of observations, a fellow data scientist advised you to use the **stochastic gradient descent** algorithm with **Polyak's heavy ball adjustment** and **Nesterov's momentum method** with **mini batch size set to $Q = 2000$ observations**.

Implement the **stochastic gradient descent method with Polyak's heavy ball adjustment and Nesterov's momentum method** to obtain the estimates of the regression coefficients.

```
# Stochastic GD w/ Polyak
SGD_Polyak <- function(y, X, startpoint, stepsize, gamma, mb_size,
  max_iter, tolerance) {
  beta <- startpoint
  beta_prev <- beta
  n <- length(y)
  indices <- 1:n

  for (i in 1:max_iter) {
    # Mini-batch selection
    idx <- sample(indices, mb_size, replace = FALSE)
    Xs <- X[idx, ]
    ys <- y[idx]

    # Gradient computation
    grad <- t(Xs) %*% (Xs %*% beta - ys)/mb_size

    # Update beta with momentum
    beta_new <- beta - stepsize * grad + gamma * (beta -
```

```

        beta_prev)

    # Convergence check
    if (sqrt(sum((beta_new - beta)^2)) < tolerance) {
        cat("Convergence achieved at iteration", i, "\n")
        break
    }

    beta_prev <- beta
    beta <- beta_new
}

return(list(beta = beta, iterations = i))
}

# Stochastic GD w/ Nesterov
SGD_Nesterov <- function(y, X, startpoint, stepsize, gamma, mb_size,
    max_iter, tolerance) {
    beta <- startpoint
    v <- rep(0, length(beta)) # Velocity term

    for (i in 1:max_iter) {
        # Mini-batch selection
        idx <- sample(1:length(y), mb_size, replace = FALSE)
        Xs <- X[idx, ]
        ys <- y[idx]

        # Lookahead
        beta_lookahead <- beta - gamma * v

        # Gradient computation
        grad <- t(Xs) %*% (Xs %*% beta_lookahead - ys)/mb_size

        # Update velocity and beta
        v <- gamma * v + stepsize * grad
        beta_new <- beta - v

        # Convergence check
        if (sqrt(sum((beta_new - beta)^2)) < tolerance) {
            cat("Convergence achieved at iteration", i, "\n")
            break
        }

        beta <- beta_new
    }

    return(list(beta = beta, iterations = i))
}

beta_init <- rep(0, p)
my_stepsize <- 0.01
max_iter <- 30000
tolerance <- 1e-06

```

```

gamma <- 0.9 # Using from hw5 - tau not needed if no bls
Q <- 2000

results_sgd_polyak <- SGD_Polyak(response, design, beta_init,
  my_stepsize, gamma, Q, max_iter, tolerance)
results_sgd_nesterov <- SGD_Nesterov(response, design, beta_init,
  my_stepsize, gamma, Q, max_iter, tolerance)

print("Results with Stochastic Polyak's Heavy Ball:")

```

```
## [1] "Results with Stochastic Polyak's Heavy Ball:"
```

```
print(results_sgd_polyak)
```

```

## $beta
##           [,1]
## [1,] 3.028006
## [2,] 3.059048
## [3,] 2.980270
## [4,] 3.003356
## [5,] 3.021010
## [6,] 2.949438
## [7,] 2.979905
## [8,] 3.061539
## [9,] 2.976702
## [10,] 2.981997
## [11,] 2.978673
## [12,] 2.973989
## [13,] 2.999511
## [14,] 3.010763
## [15,] 2.995246
## [16,] 2.994365
## [17,] 3.018136
## [18,] 3.004969
## [19,] 3.010469
## [20,] 2.998659
##
## $iterations
## [1] 30000

```

```
print("Results with Stochastic Nesterov's Momentum:")
```

```
## [1] "Results with Stochastic Nesterov's Momentum:"
```

```
print(results_sgd_nesterov)
```

```

## $beta
##           [,1]
## [1,] 3.027614
## [2,] 3.043062
## [3,] 2.966947
## [4,] 3.014354
## [5,] 3.035390
## [6,] 2.948089
## [7,] 2.979925
## [8,] 3.062871

```

```

## [9,] 2.970310
## [10,] 3.006153
## [11,] 2.966741
## [12,] 2.973524
## [13,] 2.979614
## [14,] 3.011436
## [15,] 2.991444
## [16,] 2.995718
## [17,] 3.018933
## [18,] 2.990421
## [19,] 3.006554
## [20,] 2.991004
##
## $iterations
## [1] 30000

```

Discuss how you selected the step size, and comment on the number of iterations required and the accuracy of the estimated regression coefficients compared to the least squares solution and those obtained by the standard gradient descent algorithm used in Part (b) and the gradient descent algorithm with Polyak's heavy ball adjustment and Nesterov's momentum in Part (c).

- Similar to parts a and b, I have selected a step size of 0.01 to maintain both accuracy and efficiency. This value is also a great in terms of combating overshooting with the stochastic gradient descent method, resulting in relatively accurate results. It can be observed that the maximum number of iterations is set to 30000 and a tolerance of $1e-6$, which facilitates the accuracy and efficiency of the algorithm as the values near convergence. Although the algorithm did not reach convergence within the set maximum iteration parameter, the estimated coefficients can be observed as being relatively accurate due to the values nearing the true value of 3. In comparison to the other results in part b and c, the speed of this algorithm was much faster suggesting it can handle a larger number of iterations, or even the full dataset, but with less accuracy if it were to perform within the same number of iterations as the parts b and c due to the minimal change in values between iterations.